

Jekyll Layouts & Includes — Study Roadmap

This roadmap turns our discussion into a **step-by-step learning plan** for understanding `_layouts` and `_includes` in a Jekyll project.

The goal is not to memorize files, but to build a **correct mental model** so future edits are safe, predictable, and maintainable.

Mental Model (Read First)

Jekyll rendering pipeline:

```
Markdown content (.md)
→ layout (page / post)
→ default layout
→ includes (head, header, footer, components)
```

Key idea:

- **Layouts define structure** (page skeleton)
- **Includes define reusable parts** (components)

Keep this in mind at every step.

Phase 0 — Orientation (5 minutes)

Goal: Understand how files relate without opening many of them.

- Identify these folders:
 - `_layouts`
 - `_includes`
- Know that:

- Every page eventually uses `default.html`
- Includes never stand alone

✅ Outcome: You know *where* things live and *why*.

Phase 1 — Core Page Structure (Most Important)

Step 1: `_layouts/default.html`

Why this comes first

- Every page flows through this file
- Defines what a "page" means

What to study

- `<html>` / `<body>`
- `{{ content }}` placement
- `{% include %}` calls

Questions to answer

- Where does page content go?
- Which elements are global?
- What changes would affect the whole site?

🚫 Do not edit yet — trace only.

Phase 2 — Page Type Specialization

Step 2: `_layouts/page.html`

Step 3: `_layouts/post.html`

Purpose

- Explain *why* pages and posts differ

- Add semantics, not global structure

What to look for

- How they wrap `{{ content }}`
- What logic exists only for posts (date, tags, read time)
- Which includes they use

Key insight

Layouts should stay thin. Logic belongs elsewhere.

Phase 3 — Global Includes (Site Chrome)

Step 4: `_includes/head.html`

Why early

- Critical for SEO, fonts, favicon, metadata
- Easy to break site-wide behavior

Focus on

- Meta tags
- Asset loading
- Config-driven values

Step 5: `_includes/header.html`

Step 6: `_includes/footer.html`

Step 7: `_includes/footer-end.html`

Study these together.

Look for

- Navigation structure

- Script loading patterns
- Class names used for styling

Ignore for now

- Visual styling details
- SVG complexity

Phase 4 — Reusable Content Components

Step 8: `_includes/post-card.html`

Why first among components

- High reuse (home, tags, archives)
- Encodes assumptions about post data

Study

- Expected post fields
- Conditional logic
- Markup patterns

Step 9: `_includes/pagination.html`

Focus on

- Jekyll paginator variables
- Separation of logic vs markup

You do not need to memorize this — understand the *shape*.

Step 10: `_includes/read-time.html`

Why this matters

- Shows where small logic belongs
- Demonstrates Liquid used responsibly

Key takeaway:

Small logic → include, not layout

Step 11: `_includes/social-share.html`

Study last.

Why

- Optional
- Often messy
- Easy to replace or remove later

Phase 5 — Connect HTML to SCSS (Optional but Powerful)

Once HTML structure is clear:

- Go to `_sass/5-components/`
- Match SCSS files to includes:
 - `_header.scss`
 - `_footer.scss`
 - `_pagination.scss`

This reinforces:

Markup defines structure, SCSS defines behavior and appearance

Recommended Study Pacing

Session	Focus
20–30 min	default.html
15 min	page.html , post.html
30 min	head.html
30 min	header + footer
30–45 min	components
Later	SCSS mapping

No need to do this in one sitting.

Anti-Patterns to Avoid

- ✗ Reading `_includes` before `_layouts`
- ✗ Starting with SCSS
- ✗ Trying to understand everything at once
- ✗ Editing while still building the mental model

Final Guiding Question

Whenever you feel lost, ask:

Is this file defining page structure, or a reusable part?

That single distinction will keep your Jekyll architecture clean and scalable.

This roadmap is designed to be printable, shareable, and reusable for future Jekyll projects.